Programski prevodioci

05 Sintaksna stabla

25-26

Dunja Vrbaški

Fakultet tehničkih nauka, Novi Sad

Dva osnovna stabla:

- Konkretno sintaksno stablo / stablo parsiranja
- Apstraktno sintaksno stablo

Stablo parsiranja

Struktura podataka koja odgovara gramatičkoj strukturi ulaznog stringa.

Stablo parsiranja:

- koren je početni pojam
- listovi predstavljaju simbole (terminale) ili prazan simbol
- unutrašnji čvorovi predstavljaju pojmove (neterminale)
- unutrašnji čvor koji predstavlja pojam Y i koji ima naslednike $X_1, X_2, \ldots X_n$ predstavlja pravilo izvodjenja $Y \rightarrow X_1 X_2 \ldots X_n$

assignment_stmt → ID ASSIGN num_exp SC
num_exp → exp
num_exp → num_exp PLUS exp
num_exp → num_exp MINUS exp
exp → NUM
exp → ID

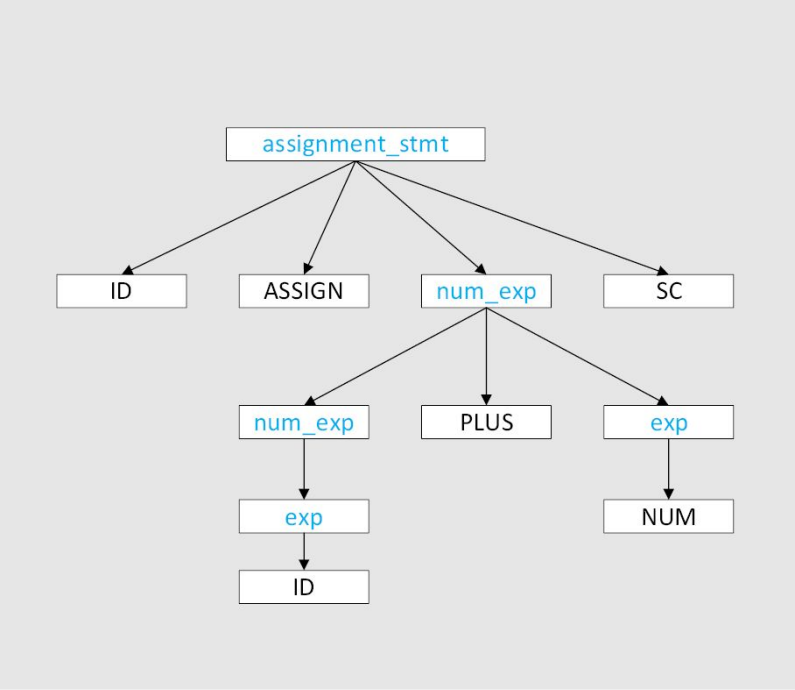
a = b + 3;
w = ID ASSIGN ID PLUS NUM SC

terminali (tokeni) – označeni velikim slovom
gramatika slična kao miniC

assignment_stmt → ID ASSIGN num_exp SC
num_exp → exp
num_exp → num_exp PLUS exp
num_exp → num_exp MINUS exp
exp → NUM
exp → ID

a = b + 3;
w = ID ASSIGN ID PLUS NUM SC

terminali (tokeni) – označeni velikim slovom
gramatika slična kao miniC



Čitajući listove s leva na desno dobijamo polazni string

Kako bismo zapisali izvođenje?

Parsiranje praktično formira stablo parsiranja
↔
Parsiranje je proces pronalaženja stabla parsiranja

top-down od korena ka listovima
bottom-up od listova ka korenu

U prevodiocu može postojati - eksplicitno ili implicitno

stablo parsiranja, stablo izvođenja, konkretno sintaksno stablo != apstraktno sintaksno stablo

U prethodnom primeru prilično jasan odabir pravila.
Nekad, prilikom izvođenja, možemo krenuti različitim “putevima”.

Posmatrajmo sledeću gramatiku:

$E \rightarrow num$
$E \rightarrow E + E$

5 + 3 + 1
w = num plus num plus num

E → num
E → E + E

5 + 3 + 1
~~w = num plus num plus num~~
w = num + num + num

Radi bolje preglednosti u nastavku ćemo “plus” pisati kao “+”

$E \rightarrow num$

$E \rightarrow E + E$

5 + 3 + 1

w = num + num + num

$$
\begin{aligned}
& E \\
\Rightarrow\ & E + E \\
\Rightarrow\ & num + E \\
\Rightarrow\ & num + E + E \\
\Rightarrow\ & num + num + E \\
\Rightarrow\ & num + num + num
\end{aligned}
$$

$$
\begin{aligned}
& E \\
\Rightarrow\ & E + E \\
\Rightarrow\ & E + num \\
\Rightarrow\ & E + E + num \\
\Rightarrow\ & E + num + num \\
\Rightarrow\ & num + num + num
\end{aligned}
$$

Leftmost derivation (najlevlje izvođenje, izvođenje s leva)
Izvođenje u kom se uvek primenjuje pravilo na prvi pojam sa leve strane

Rightmost derivation (najdešnje izvođenje, izvođenje s desna)
Izvođenje u kom se uvek primenjuje pravilo na prvi pojam sa desne strane

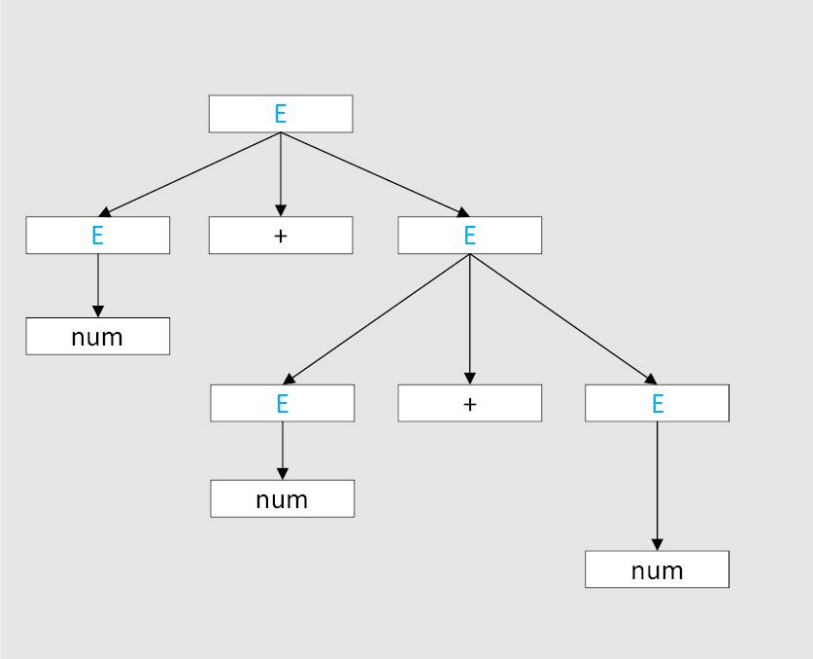
Postoje izvođenja koja ne prate ni jedno od ove dve heuristike.

Leftmost

E → num
E → E + E

5 + 3 + 1
w = num + num + num

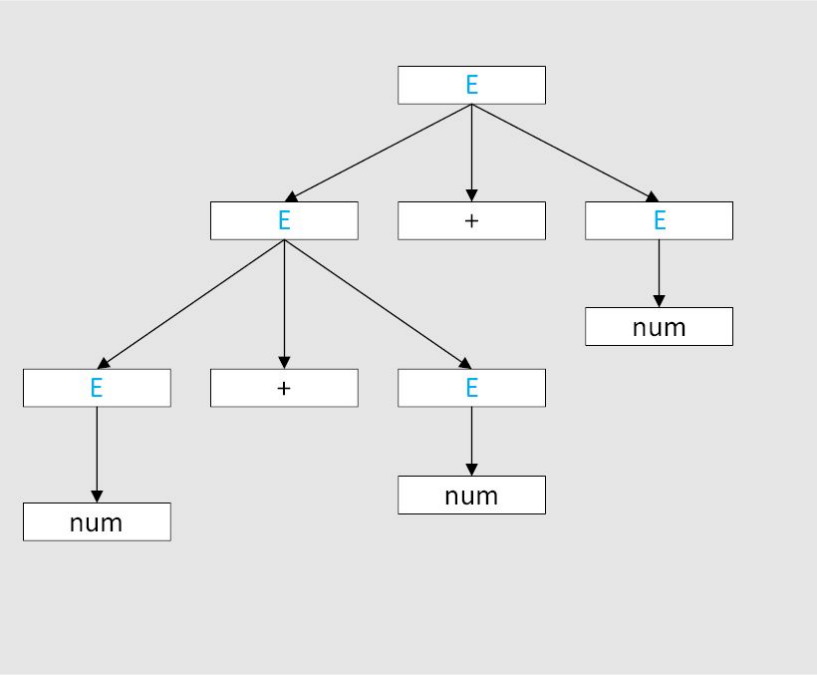
E
⇒ E + E
⇒ num + E
⇒ num + E + E
⇒ num + num + E
⇒ num + num + num



Rightmost

E → num
E → E + E

5 + 3 + 1
w = num + num + num



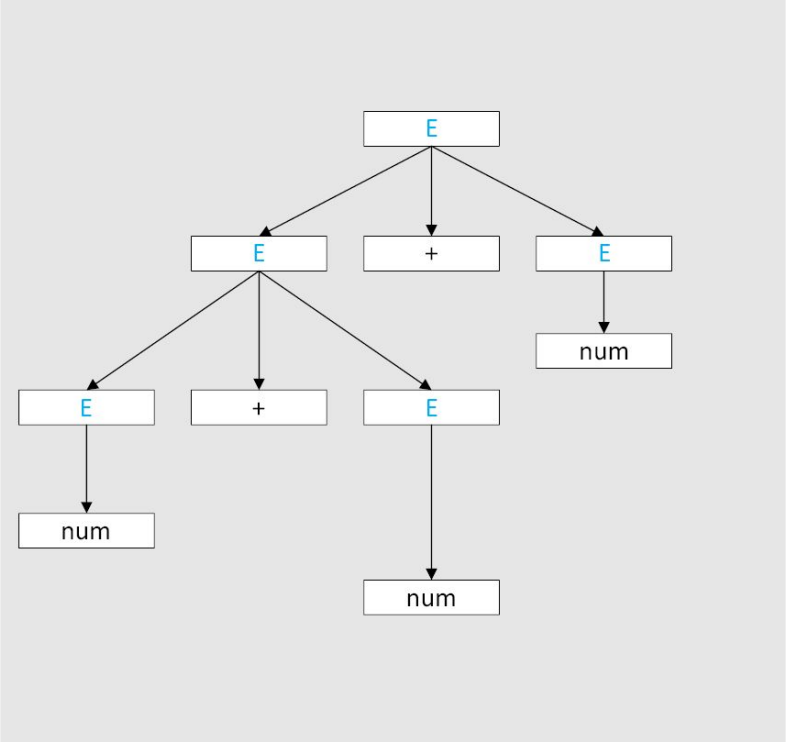
E
⇒ E + E
⇒ E + num
⇒ E + E + num
⇒ E + num + num
⇒ num + num + num

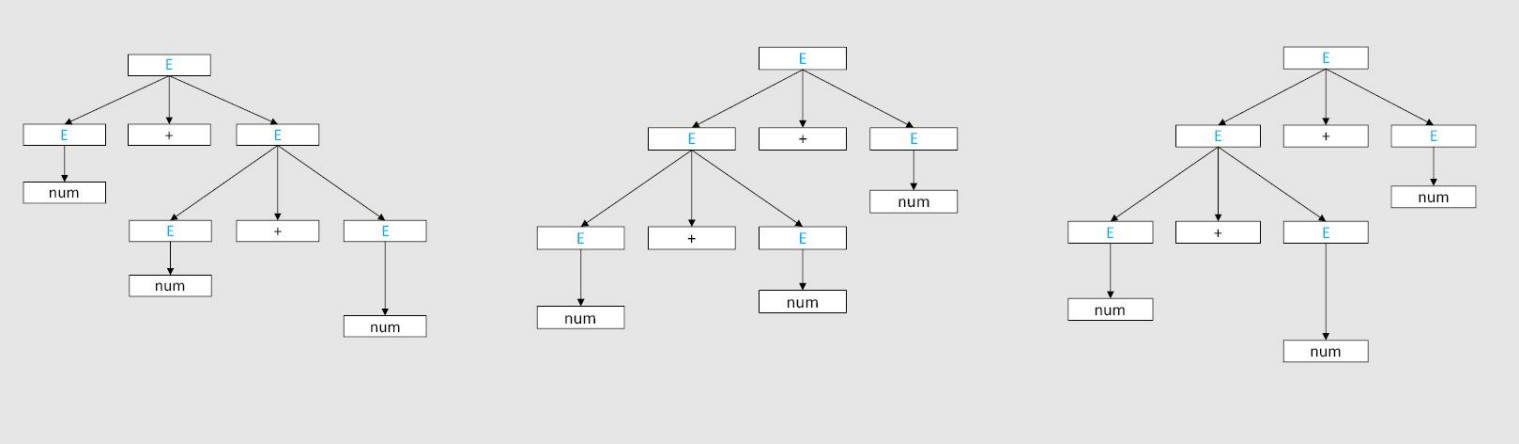
Ni L ni R

E → num
E → E + E

5 + 3 + 1
w = num + num + num

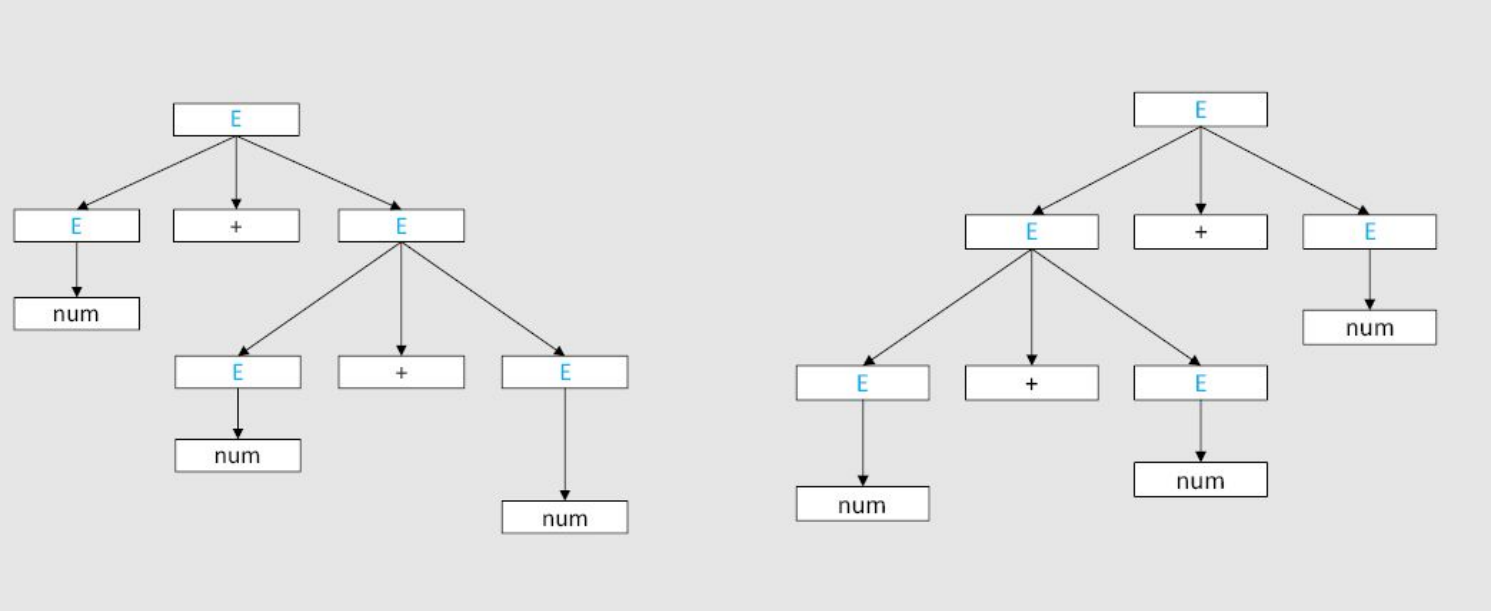
E
⇒ E + E
⇒ E + num
⇒ E + E + num
⇒ num + E + num
⇒ num + num + num





Drugo i treće stablo: zapravo ne postoji informacija o redosledu.

Na slikama je modelovan redosled (grafički spuštanjem listova), ali stablo, suštinski, nema tu informaciju (dužine strelica).



Može se primetiti da stablo utiče na asocijativnost operacija
num + (num + num)
(num + num) + num

5 + 3 + 1

Ako je operator + levo asocijativan operand 3 će biti preuzet od strane levog + operatora
(5 + 3) + 1

Ako je operator + desno asocijativan operand 3 će bit preuzet od strane desnog + operatora
5 + (3 + 1)

5 + 3 + 1

Obratiti pažnju: ovde nije morao biti operator sabiranja niti su operandi nužno numeričke vrednosti.

Može biti značajno kod evaluacije.

Iako se trenutno ne bavimo evaluacijom, zanima nas samo struktura, razmisliti:
šta ako i kad bude trebala evaluacija ovog izraza (izračunavanje)?

Već sada to moramo imati rešeno.

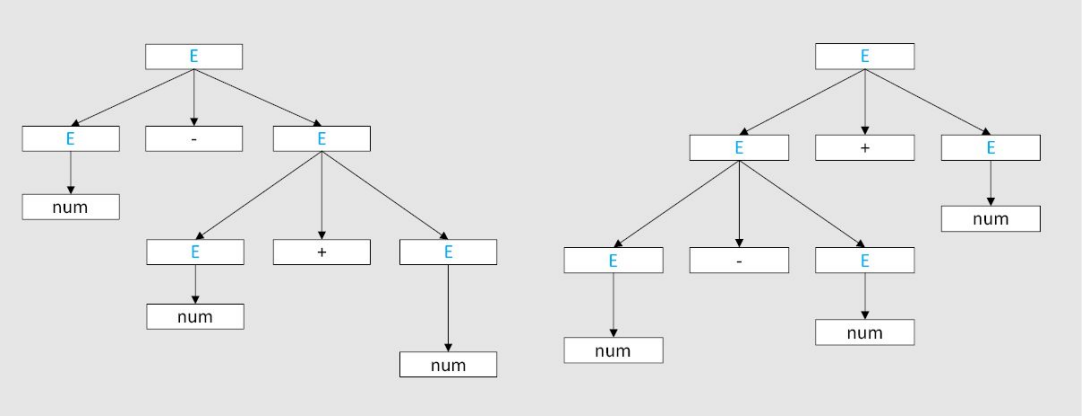
Posmatrajmo sledeću gramatiku

E → num
E → E + E
E → E - E

5 - 3 + 1
w = num - num + num

E → num
E → E + E
E → E - E

5 - 3 + 1
w = num - num + num



5 – (3 + 1)

(5 – 3) + 1

Obratiti pažnju: Gramatika nema zagrade.
Ovde, ispod slike, su iskorišćene samo kao prikaz potencijalnog redosleda evaluacije.

Različita stabla parsiranja mogu odgovarati istom stringu.

Kažemo da je gramatika dvosmislena (višeznačna) ako za neki polazni string postoje bar dva različita stabla parsiranja.

Ako svaki string ima jedinstveno stablo parsiranja onda svaki string ima i jedinstveno najlevlje izvođenje i jedinstveno najdešnje izvođenje.

Ne postoji univerzalni algoritam za detekciju dvosmislene gramatike (odlučivanje) ili njenog pretvaranja u gramatiku koja nije dvosmislena.

Kako da rešimo?

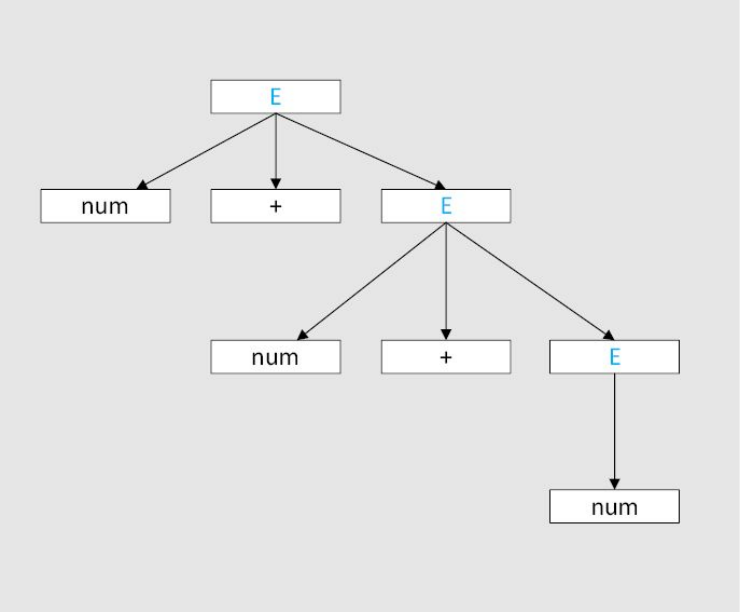
- Možemo promeniti gramatiku. Dvosmislenost je osobina gramatike – ne jezika!
- Koristeći neke mehanizme/ideje

~~E → num~~
~~E → E + E~~

E → num
E → num + E

5 + 3 + 1
w = num + num + num

E
⇒ num + E
⇒ num + num + E
⇒ num + num + num



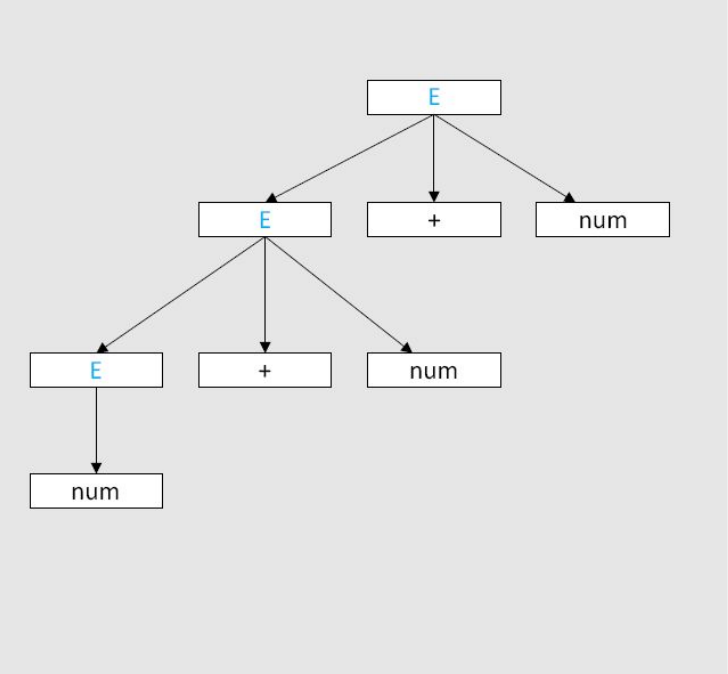
Obratiti pažnju: definisano kao lista

~~E → num~~
~~E → E + E~~

E → num
E → E + num

5 + 3 + 1
w = num + num + num

E
⇒ E + num
⇒ E + num + num
⇒ num + num + num



$E \rightarrow num$
$E \rightarrow E + E$

dvosmislena

$E \rightarrow num$
$E \rightarrow num + E$

desna rekurzija

$E \rightarrow num$
$E \rightarrow E + num$

leva rekurzija

Dvosmislenost jeste problem, vrsta rekurzije može odrediti stablo, međutim vrsta rekurzije koja se pojavljuje može biti problem za određenu vrstu parsiranja.

Posmatrajmo sledeću gramatiku:

(na trenutak zaboravimo na to što imamo dvostruku rekurziju)

$E \rightarrow num$

$E \rightarrow E + E$

$E \rightarrow E * E$

5 + 3 * 2

w = num + num * num

1. E → num
2. E → E + E
3. E → E * E

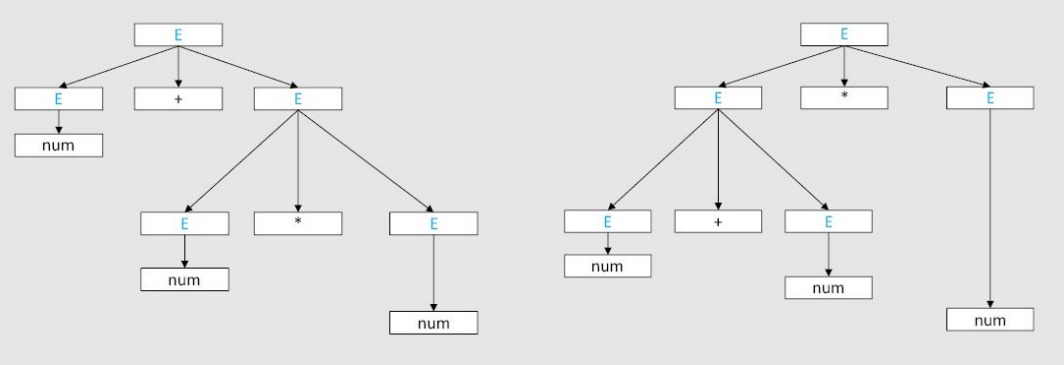
5 + 3 * 2
w = num + num * num

leftmost + prvo pravilo 2

E
⇒ E + E
⇒ num + E
⇒ num + E * E
⇒ num + num * E
⇒ num + num * num

leftmost + prvo pravilo 3

E
⇒ E * E
(ne može se menjati sa num jer nećemo stići do izraza koji tražimo)
⇒ E + E * E
⇒ num + E * E
⇒ num + num * E
⇒ num + num * num



5 + (3 *2)

(5 + 3) * 2

Potrebno je voditi računa i o prioritetu operatora.

Moramo obezbediti asocijativnost.
Moramo obezbediti prioritet.

Kako da rešimo?

- Možemo promeniti gramatiku
- Možemo možda nekako definisati prioritet (forsirati određena pravila).

Ideja:
5 + 3 * 2 + 4 * 1 ...
Posmatramo kao sabirke

~~1. E → num~~
~~2. E → E + E~~
~~3. E → E * E~~

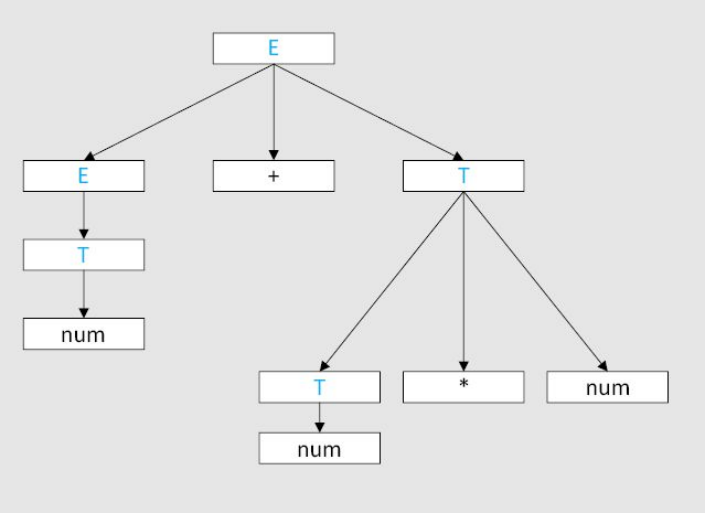
1. E → E + T
2. E → T
3. T → T * num
4. T → num

5 + 3 * 2
w = num + num * num

Obratiti pažnju: definisano kao liste

leftmost

E
⇒ E + T
⇒ T + T
⇒ num + T
⇒ num + T * num
⇒ num + num * num



- miniC ima samo + i –
- Ako se dodaje * i / - treba razmisliti o ovome

- promena gramatike: dobra opcija

- bison:
  - operator naveden kasnije ima veći prioritet
  - operatori navedeni u istom redu imaju isti prioritet
  - postoji posebni mehanizmi za definisanje asocijativnosti i prioriteta (u nastavku)

Dangling else problem

stmt → assign_stmt
stmt → if_stmt
stmt → while_stmt
stmt → …

if_stmt → IF exp stmt
if_stmt → IF exp stmt ELSE stmt

ILI (kraće, liči na bison)

stmt	→	if_stmt
if_stmt	→	iF exp stmt
	|	IF exp stmt ELSE stmt

stmt → assign_stmt
stmt → if_stmt
stmt → …

if_stmt → IF exp stmt
if_stmt → IF exp stmt ELSE stmt

```
if (a != b) if (a > b) b = a; else a = b;
```

Kako bismo želeli da se interpretira?

ILI (kraće, liči na bison)

stmt	→	if_stmt
if_stmt	→	iF exp stmt
	|	IF exp stmt ELSE stmt

stmt → assign_stmt
stmt → if_stmt
stmt → …

if_stmt → IF exp stmt
if_stmt → IF exp stmt ELSE stmt

ILI (kraće, liči na bison)

stmt	→	if_stmt
if_stmt	→	iF exp stmt
	|	IF exp stmt ELSE stmt

```
if (a != b) if (a > b) b = a; else a = b;
```

Kako bismo želeli da se interpretira?

```
if (a != b)
   if (a > b)
      b = a;
   else
      a = b;
```

```
if (a != b)
   if (a > b)
      b = a;
else
    a = b;
```

stmt	→	if_stmt
if_stmt	→	iF exp stmt
	|	IF exp stmt ELSE stmt

```
if (a != b) if (a > b) b = a; else a = b;
```

w= IF LP ID ROP ID RP IF LP ID ROP ID ID ASGN ID SC ELSE ID ASGN ID SC

if_stmt
⇒ **IF exp stmt**
⇒ IF … stmt
⇒ IF … if_stmt
⇒ IF … IF exp stmt ELSE stmt
⇒ IF … IF … stmt ELSE stmt
⇒ IF … IF … … ELSE stmt
⇒ IF … IF … … ELSE …

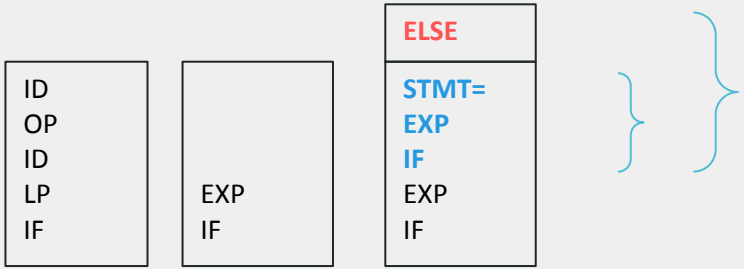
if_stmt
⇒ **IF exp stmt ELSE stmt**
⇒ IF … stmt ELSE stmt
⇒ IF … if_stmt ELSE stmt
⇒ IF … IF exp stmt ELSE stmt
⇒ IF … IF … stmt ELSE stmt
⇒ IF … IF … … ELSE stmt
⇒ IF … IF … … ELSE …

Koje izvođenje je "tačno"? I zašto je ovo pod navodnicima?

stmt	→	if_stmt
if_stmt	→	iF exp stmt
	|	IF exp stmt ELSE stmt

```
if (a != b) if (a > b) b = a; else a = b;
```

w= IF LP ID ROP ID RP IF LP ID ROP ID ID ASGN ID SC ELSE ID ASGN ID SC



Podrazumevano, bison generiše LALR(1) parser

1 lookahead token

left-to-right scanning of the input for constructing a rightmost derivation in reverse

Ukoliko postoji višeznačnost javljaju se konflikti.
Potencijalno se, u datom stanju, može izvršiti više akcija.

shift-reduce konflikti
Kada, posmatrajući LA token, može da se izvrši i jedna i druga akcija

reduce-reduce konflikti
Kada postoji više pravila koja se mogu primeniti na ulazni niz.

Postoji podrazumevano razrešenje konflikta u bison-u.

shift-reduce:

- Prednost se daje shift akciji.
- Ukoliko tokeni i pravila imaju definisan prioritet onda se podrazumevana akcija menja.

reduce-reduce:

- Prednost se daje reduce pravilu koje je ranije navedeno.

Za terminale postoje sledeće deklaracije (*navode se umesto %token*)

```
%left
%right
%nonassoc
%precedence
```

`%left` levo asocijativan

`%right` desno asocijativan

`%nonassoc` nije asocijativan
a OP b OP c - sintaksna greška

`%precedence` definiše samo prioritet, ne definiše asocijativnost
(Za slučajeve kada asocijativnost nije potrebna i kad predstavlja višak informacija i zapravo može i da napravi/sakrije drugi problem.)

+ ranije navedeni ima manji prioritet; zajedno navedeni imaju isti prioritet

```
%left '<' '>' '='
%left '+' '-'
%left '*' '/'
```

Pravila preuzimaju prioritet poslednjeg tokena.

Može se eksplicitno promeniti prioritet pravila dodeljivanjem prioriteta određenog tokena

```
%prec token
```

```
%left '+' '-'
%left '*' '/'
%left UMINUS
```

```
exp : exp '+' exp
    | exp '-' exp
    | exp '*' exp
    | exp '/' exp
    | '-' exp %prec UMINUS
    | /* ...  */
    ;
```

UMINUS je token, ali se nikad ne pojavljuje kao prepoznati token pri skeniranju.
Dodat je samo zbog razrešenja.
Data gramatika se može transformisati tako da nema potrebe za deklaracijama.

Primer. Obratiti pažnju. Koji je poslednji token i koji prioritet ima.

```
exp : exp '+' exp
```

```
exp : exp '+' exp ';'
```

shift-reduce konflikti

Kada, posmatrajući LA token, može da se izvrši i jedna i druga akcija

5 + 2 + 3	□	(5 + 2) + 3	potreban reduce
5 + 2 * 3	□	5 + (2 * 3)	potreban shift
5 * 2 + 3	□	(5 * 2) + 3	potreban reduce

Razrešavanje

Reduce →

- Ako je prioritet pravila veći od prioriteta LA tokena.
- Ako imaju isti prioritet i asocijativnost pravila je left.

Shift → u ostalim slučajevima

- Ako je prioritet pravila manji od prioriteta LA tokena
- Ako imaju isti prioritet i asocijativnost pravila nije left.
- Ako token ili pravilo nemaju prioritet.

Razrešavanje

Reduce →
- Ako je prioritet pravila veći od prioriteta LA tokena.
- Ako imaju isti prioritet i asocijativnost pravila je left.

Shift → u ostalim slučajevima
- Ako je prioritet pravila manji od prioriteta LA tokena
- Ako imaju isti prioritet i asocijativnost pravila nije left.
- Ako token ili pravilo nemaju prioritet.

```
%left '+' '-'
%left '*' '/'
%left UMINUS
```

```
exp : exp '+' exp
    | exp '-' exp
    | exp '*' exp
    | exp '/' exp
    | '-' exp %prec
UMINUS
    | /* ...  */
    ;
```

5 + 2 + 3 □ (5 + 2) + 3 potreban reduce

5 + 2 * 3 □ 5 + (2 * 3) potreban shift

5 * 2 + 3 □ (5 * 2) + 3 potreban reduce

```
%nonassoc ONLY_IF
%nonassoc _ELSE

%%


if_statement
  : if_part %prec ONLY_IF
  | if_part _ELSE statement
  ;

if_part
  : _IF _LPAREN rel_exp _RPAREN statement
  ;
```

Razrešavanje

Reduce →
- Ako je prioritet pravila veći od prioriteta LA tokena.
- Ako imaju isti prioritet i asocijativnost pravila je left.

Shift → u ostalim slučajevima
- Ako je prioritet pravila manji od prioriteta LA tokena
- Ako imaju isti prioritet i asocijativnost pravila nije left.
- Ako token ili pravilo nemaju prioritet.

```
IF LPAREN rel_exp _RPAREN IF LPAREN rel_exp RPAREN
statement ELSE statement
```

Kad ELSE bude LA:
- može reduce if_part → if_statement / può shift ELSE
- pravilo ima proiritet kao ONLY_IF token
- → pravilo za reduce ima manji prioritet od tokena za shift
- → uradi shift

```
%nonassoc ONLY_IF
%nonassoc _ELSE

%%


if_statement
  : if_part %prec ONLY_IF
  | if_part _ELSE statement
  ;

if_part
  : _IF _LPAREN rel_exp _RPAREN statement
  ;
```

reduce-reduce konflikti

Kada postoji više pravila koja se mogu primeniti na ulazni niz.

Razrešavanje

Prednost se daje reduce akciji čije pravilo je prvo navedeno.

- Podrazumevane akcije mogu biti neodgovarajuće.
- Posebno obratiti pažnju na reduce-reduce konflikte.
- Ukazuje na ozbiljan problem u gramatici.
- Bolje proučiti i izmeniti gramatiku.

Pitanja i zadaci

Koji operator je desno asocijativan?

Dodati unarni minus u miniC.

Da li se IF problem moze resiti uz pomoc operatora %left, %right ili %precedence?

Da li se IF problem moze resiti promenom gramatike?